

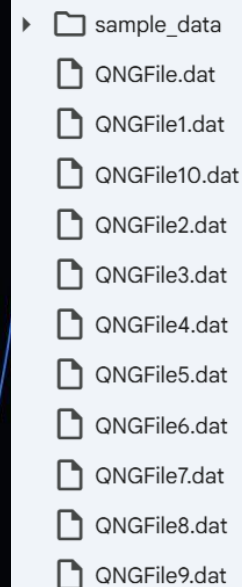


Symulator QKD (BB84 i BBM92)

Grzegorz Ciapa 278639

Źródło losowości

- Zakładamy, że wczytywany **plik .dat to twardy zapis rzeczywistych pomiarów zjawisk kwantowych**, co czyni dane realnie losowym ciągiem bitów.
- Zakładamy, że **plik to jednowymiarowy, niesformatowany ciąg czystych bitów** i wymusza jego otwarcie w trybie odczytu binarnego (**rb**).
- Wprowadzono **założenie losowego przesunięcia wskaźnika** przy starcie (**seek**) – każde uruchomienie kodu korzysta z innej puli kwantowych bitów.
- Kod zakłada **optymalizację** - pobiera całe pakiety (1 bajt = 8 bitów zdarzeń kwantowych) do pamięci podręcznej RAM (**zmienna buffer**), a następnie za pomocą operatorów matematycznych wydziela z niego pojedyncze wartości 0 i 1.



```
sample_data
├── QNGFile.dat
├── QNGFile1.dat
├── QNGFile10.dat
├── QNGFile2.dat
├── QNGFile3.dat
├── QNGFile4.dat
├── QNGFile5.dat
├── QNGFile6.dat
├── QNGFile7.dat
├── QNGFile8.dat
└── QNGFile9.dat
```

```
import os
import random
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector

class QRNGFileGenerator:
    def __init__(self, filepath):
        self.filepath = filepath
        self.file = open(filepath, 'rb')

        # Przesunięcie wskaźnika
        losowy_start = random.randint(0, 1024)
        self.file.seek(losowy_start)

        self.buffer = 0
        self.bit_pos = 0

    def get_next_bit(self):
        if self.bit_pos == 0:
            byte = self.file.read(1)
            if not byte:
                self.file.seek(0)
                byte = self.file.read(1)

            self.buffer = int.from_bytes(byte, byteorder='big')
            self.bit_pos = 8

        self.bit_pos -= 1
        bit = (self.buffer >> self.bit_pos) & 1
        return bit

    def close(self):
        self.file.close()
```

Symulacja źródła fotonów BB84

- Kodowanie informacji - Alicja koduje dwa parametry dla każdego fotonu: **wartość bitu** oraz **bazę polaryzacji**
- Losowość kwantowa - **wybór bazy i bitu jest nieprzewidywalny** - program dwukrotnie odwołuje się do modułu `qrng` dla każdego fotonu.
- Niedoskonałości - **Parametr `source_error_rate`** pozwala na symulację rzeczywistego sprzętu, który może z pewnym prawdopodobieństwem zniekształcić stan fotonu już w momencie jego generacji.
- Sposób określania **stanów dla bitów**:
 - Za **stan początkowy** fotonu uznajemy **stan $|0\rangle$**
 - Dla **bitu 1** działamy na stan początkowy bramką Pauliego (X) i otrzymujemy **stan $|1\rangle$**
 - Dla bitu **0** przypisujemy **stan $|0\rangle$**
- Sposób określania **baz polaryzacji**:
 - **Bazie prostej** przypisujemy wartość = 0, przez którą **foton przechodzi bez zmian**
 - **Bazie skośnej** przypisujemy wartość = 1 i nakładamy na nią **bramkę Hadamarda** – foton w tej bazie zostaje wprowadzony w stan superpozycji (obracamy bazę o 45 stopni)

```
# Źródło fotonów idealne BB84
```

```
def alicja_przygotuj_fotony(ilosc_fotonow, qrng, source_error_rate=0.01):
    obwody_kwantowe = []
    bity_alicji = []
    bazy_alicji = []
    for _ in range(ilosc_fotonow):
        wartosc_bitu = qrng.get_next_bit()
        baza_pomiarowa = qrng.get_next_bit()
        bity_alicji.append(wartosc_bitu)
        bazy_alicji.append(baza_pomiarowa)

        qc = QuantumCircuit(1)
        if wartosc_bitu == 1: qc.x(0)
        if baza_pomiarowa == 1: qc.h(0)

        if random.random() < source_error_rate:
            qc.x(0)
        obwody_kwantowe.append(qc)
    return obwody_kwantowe, bity_alicji, bazy_alicji
```

Symulacja źródła fotonów BBM92

- **Układ opiera się na splątaniu kwantowym** - laser generuje parę splątanych fotonów. Jeden z nich zostaje u Alicji, a drugi leci do Boba.
- Parametr **source_error_rate** pozwala na symulację rzeczywistego sprzętu, który z pewnym prawdopodobieństwem może zniekształcić stan foton.
- Sposób generowania stanu splątanego:
 - Za stan początkowy pary fotonów uznajemy **stan | 00 ⟩**
 - Na pierwszy foton (Alicji) nakładana jest **bramka Hadamarda (H)**, co wprowadza go w stan idealnej superpozycji.
 - Następnie używamy dwukubitowej **bramki CNOT**, która trwale wiąże foton drugi (Boba) z wynikiem pomiaru fotonu pierwszego.
- Sposób określania baz polaryzacji i pomiaru:
 - Bazie prostej przypisujemy wartość = 0, przed pomiarem foton Alicji przechodzi bez zmian.
 - Bazie skośnej przypisujemy wartość = 1 i nakładamy na nią bramkę Hadamarda – foton przed detekcją zostaje obrócony o 45 stopni.
- **Kolaps stanu** - pomiar fotonu u Alicji za pomocą instrukcji `measure` natychmiastowo determinuje stan drugiego fotonu, który w skorelowanej formie leci do Boba w nowym obwodzie.

Źródło fotonów BBM92

```
def zdrojlo_splatane_i_alicja(ilosc_par, qrng, source_error_rate=0.01):
    bity_alicji = []
    bazy_alicji = []
    obwody_w_locie = []

    for _ in range(ilosc_par):
        baza_a = qrng.get_next_bit()
        bazy_alicji.append(baza_a)

        # Para splatana
        qc = QuantumCircuit(2) # 2 fotony
        qc.h(0)                 # Superpozycja
        qc.cx(0, 1)             # Bramka CNOT - splatanie

        if random.random() < source_error_rate:
            qc.x(1)

        if baza_a == 1:
            qc.h(0) # Alicja obraca bazę na swoim fotonie

        sv = Statevector(qc)

        # kolaps fotonu Boba
        bit_str, sv_zapadniety = sv.measure([0])
        bity_alicji.append(int(bit_str))

        # Nowy obwód
        qc_kontynuacja = QuantumCircuit(2)
        qc_kontynuacja.initialize(sv_zapadniety, [0, 1])
        obwody_w_locie.append(qc_kontynuacja)

    return obwody_w_locie, bity_alicji, bazy_alicji
```

Symulacja nieidealnego źródła fotonów BB84

Różnice między idealnym źródłem fotonów dla BB84

Emisja wielofotonowa:

- **Parametr `p_multi`** symuluje fizyczne ograniczenia rzeczywistych laserów telekomunikacyjnych - określonym prawdopodobieństwem laser emituje paczkę dwóch identycznych fotonów zamiast jednego (zapisywane w `lista_multi`)
- Podatność na atak PNS

```
# Źródło nieidealne

def alicja_przygotuj_fotony(ilosc_fotonow, qrng, source_error_rate=0.01, p_multi=0.1):
    obwody_kwantowe = []
    bity_alicji = []
    bazy_alicji = []
    lista_multi = []

    for _ in range(ilosc_fotonow):
        wartosc_bitu = qrng.get_next_bit()
        baza_pomiarowa = qrng.get_next_bit()
        bity_alicji.append(wartosc_bitu)
        bazy_alicji.append(baza_pomiarowa)

        wielokrotnosc = 2 if random.random() < p_multi else 1
        lista_multi.append(wielokrotnosc)

        qc = QuantumCircuit(1)
        if wartosc_bitu == 1: qc.x(0)
        if baza_pomiarowa == 1: qc.h(0)

        if random.random() < source_error_rate:
            qc.x(0)
        obwody_kwantowe.append(qc)

    return obwody_kwantowe, bity_alicji, bazy_alicji, lista_multi
```


Symulacja kanału kwantowego

- **Parametr `channel_error_rate`** określa prawdopodobieństwo wystąpienia uszkodzenia dla każdego pojedynczego fotonu (**domyślnie ustawione na 3%** - naturalne zniekształcenia sygnału w prawdziwym włóknie optycznym).
- Sposób implementacji **błędów kwantowych**:
 - Program dla każdego fotonu niezależnie losuje szansę na wystąpienie błędu. Jeśli wystąpi, przypisuje mu z równym prawdopodobieństwem jedną z trzech fundamentalnych **bramek Pauliego (X, Y, Z)**.
 - **Błąd typu X (Bit-flip)** – symuluje fizyczne odwrócenie polaryzacji (bramka X), zamieniając klasyczne wartości 0 na 1 i vice versa.
 - **Błąd typu Z (Phase-flip)** – wprowadza błąd fazy (bramka Z), który jest obojętny dla bazy prostej, ale całkowicie niszczy informację zakodowaną w bazie skośnej (superpozycji).
 - **Błąd typu Y (Bit-phase-flip)** – stanowi złożenie obu powyższych zniekształceń naraz (bramka Y).
- Kanał **nie dokonuje pomiaru** przelatujących fotonów

Kanał kwantowy

```
def kanal_kwantowy(obwody_kwantowe, channel_error_rate=0.03):
    obwody_po_przejsciu = []
    for qc in obwody_kwantowe:
        qc_kanal = qc.copy()
        if random.random() < channel_error_rate:
            rodzaj_bledu = random.choice(['X', 'Y', 'Z'])
            if rodzaj_bledu == 'X': qc_kanal.x(0)
            elif rodzaj_bledu == 'Y': qc_kanal.y(0)
            elif rodzaj_bledu == 'Z': qc_kanal.z(0)
        obwody_po_przejsciu.append(qc_kanal)
    return obwody_po_przejsciu
```

Symulacja kanału kwantowego - pomiar

- Dla każdego fotonu program odwołuje się do modułu **qrng**, aby w sposób całkowicie losowy i nieprzewidywalny wybrać bazę pomiarową.
- **Parametr detector_error_rate** symuluje błędy rzeczywistych detektorów jednofotonowych. **Domyślna wartość 1%** oznacza, że z takim prawdopodobieństwem detektor zarejestruje fałszywy impuls, co w kodzie realizowane jest przez **operator XOR**, odwracający poprawny wynik.
- **Sposób realizacji pomiaru:**
 - Wybór bazy – jeśli wylosowana wartość wynosi 0, Bob mierzy w **bazie prostej** (brak zmian w obwodzie). Jeśli wynosi 1, mierzy w **bazie skośnej** i nakłada **bramkę Hadamarda (H)**, aby "obrócić" stan superpozycji z powrotem do bazy obliczeniowej.
 - Kolaps funkcji falowej – **instrukcja Statevector.measure()** symuluje interakcje fotonu z detektorem. Powoduje to natychmiastowe zredukowanie stanu kwantowego do konkretnej wartości klasycznej (0 lub 1).
 - Rejestracja danych – wynik pomiaru (bit) oraz użyta baza są zapisywane w lokalnej pamięci Boba (**wyniki_boba, bazy_boba**), co umożliwi późniejszą fazę uzgadniania klucza (sifting).

Pomiar

```
def bob_pomiar(obwody_po_kanale, qrng, detector_error_rate=0.01):  
    bazy_boba = []  
    wyniki_boba = []  
    for qc in obwody_po_kanale:  
        baza_pomiarowa = qrng.get_next_bit()  
        bazy_boba.append(baza_pomiarowa)  
  
        qc_pomiar = qc.copy()  
        if baza_pomiarowa == 1: qc_pomiar.h(0)  
  
        wektor_stanu = Statevector(qc_pomiar)  
        zmierzony_wynik_str, _ = wektor_stanu.measure()  
        bit = int(zmierzony_wynik_str)  
  
        if random.random() < detector_error_rate:  
            bit ^= 1  
  
        wyniki_boba.append(bit)  
    return wyniki_boba, bazy_boba
```

Atak Ewy BB84

- Parametr **tryb** definiuje częstotliwość ingerencji w kanał
 - Tryb **probabilistyczny** – atakuje pojedyncze fotony z zadanyam prawdopodobieństwem (np. 10%).
 - Tryb **sekwencyjny** – włącza ciągły podsłuch od ściśle określonego momentu (kroku).
 - Tryb **interwałowy** – podsłuchuje cyklicznie (np. dokładnie co 2, 4 lub n-ty foton).
- Strategie pomiarowe - parametr **strategia** określa, w jaki sposób Ewa dobiera bazę do pomiaru skradzionego fotonu:
 - Strategia **losowa** – Ewa zachowuje się identycznie jak Bob, używając modułu **qrng** do odgadywania bazy w ciemno.
 - Strategie **prosta** lub **skośna** – Ewa sztywno używa tylko jednej bazy, odczytując fotony zgodne z jej ustawieniem, ale systematycznie niszcząc stany zakodowane w przeciwnej bazie.
- Sposób realizacji ataku (**Intercept-Resend**):
 - **Pominięcie** – jeśli algorytm decyduje o braku podsłuchu, foton jest przepuszczany nienaruszony (kopiowanie oryginalnego obwodu).
 - **Przechwycenie (Intercept)** – Ewa nakłada wybraną bazę (ewentualna bramka H) i wymusza **kolaps funkcji falowej** (instrukcja `measure`). Zgodnie z zakazem klonowania w mechanice kwantowej, oryginalny stan fotonu zostaje zniszczony.
 - **Wysłanie (Resend)** – Ewa tworzy **zupełnie nowy obwód kwantowy** (`QuantumCircuit(1)`). Koduje na nim odczytaną przez siebie wartość i bazę, a następnie wysyła tak spreparowaną "podróbkę" w dalszą drogę do kanału kwantowego.

```
# Podsłuch Ewy

def podsłuch_ewy(obwody_kwantowe, qrng, strategia='losowa', tryb='probabilistyczny', prawdopodobienstwo=0.1, start_krok=0, interwal=2):
    obwody_po_ewie = []
    bity_ewy = []
    bazy_ewy = []

    for i, qc in enumerate(obwody_kwantowe):
        podsłuchuje = False
        if tryb == 'probabilistyczny':
            if random.random() < prawdopodobienstwo: podsłuchuje = True
        elif tryb == 'sekwencyjny':
            if i >= start_krok: podsłuchuje = True
        elif tryb == 'interwałowy':
            if interwal > 0 and (i + 1) % interwal == 0: podsłuchuje = True

        if not podsłuchuje:
            obwody_po_ewie.append(qc.copy())
            bity_ewy.append(None)
            bazy_ewy.append(None)
            continue

        if strategia == 'prosta': baza = 0
        elif strategia == 'skosna': baza = 1
        else: baza = qrng.get_next_bit()

        bazy_ewy.append(baza)

        qc_ewa = qc.copy()
        if baza == 1: qc_ewa.h(0)
        wektor_stanu = Statevector(qc_ewa)
        zmierzony_wynik_str, _ = wektor_stanu.measure()
        zmierzony_bit = int(zmierzony_wynik_str)
        bity_ewy.append(zmierzony_bit)

        nowy_foton = QuantumCircuit(1)
        if zmierzony_bit == 1: nowy_foton.x(0)
        if baza == 1: nowy_foton.h(0)
        obwody_po_ewie.append(nowy_foton)

    return obwody_po_ewie, bity_ewy, bazy_ewy
```


Atak Ewy BBM92

- Sposób realizacji ataku na stan splątany (**Intercept-Resend**):
 - Ewa dobiera bazę do pomiaru losowo (przy użyciu **qrng**) lub stosuje strategię sztywną (**prosta / skośna**).
 - **Zniszczenie splątania** – Ewa przechwytuje wyłącznie **foton lecący do Boba (kubit 1)**. Nakłada na niego wybraną przez siebie bazę (**qc_ewa.h(1)**) i wymusza pomiar (**measure([1])**). Ten akt interakcji powoduje przedwczesny kolaps funkcji falowej i bezpowrotnie **niszczy nielokalną korelację** między cząstką Alicji i Boba.
 - Ewa inicjuje **zupełnie nowy obwód kwantowy** (**QuantumCircuit(2)**). Koduje odczytaną przez siebie wartość klasyczną bezpośrednio na rejestrze Boba (**x(1), h(1)**) i wysyła tak spreparowany, sztuczny foton w dalszą drogę (**Resend**).

```
# Podśluch Ewy BBM92

def podsluch_ewy(obwody_kwantowe, qrng, strategia='losowa', tryb='probabilistyczny', prawdopodobienstwo=0.1, start_krok=0, interwal=2):
    obwody_po_ewie = []
    bity_ewy = []
    bazy_ewy = []

    for i, qc in enumerate(obwody_kwantowe):
        podsluchuje = False
        if tryb == 'probabilistyczny':
            if random.random() < prawdopodobienstwo: podsluchuje = True
        elif tryb == 'sekwencyjny':
            if i >= start_krok: podsluchuje = True
        elif tryb == 'interwalowy':
            if interwal > 0 and (i + 1) % interwal == 0: podsluchuje = True

        if not podsluchuje:
            obwody_po_ewie.append(qc.copy())
            bity_ewy.append(None)
            bazy_ewy.append(None)
            continue

        if strategia == 'prosta': baza = 0
        elif strategia == 'skosna': baza = 1
        else: baza = qrng.get_next_bit()

        bazy_ewy.append(baza)

        # Ewa mierzy foton lecący do Boba
        qc_ewa = qc.copy()
        if baza == 1: qc_ewa.h(1)
        sv = Statevector(qc_ewa)
        bit_str, _ = sv.measure([1])
        zmierzony_bit = int(bit_str)
        bity_ewy.append(zmierzony_bit)

        # Ewa przerywa splątanie i wysyła spreparowany foton
        qc_new = QuantumCircuit(2)
        if zmierzony_bit == 1: qc_new.x(1)
        if baza == 1: qc_new.h(1)
        obwody_po_ewie.append(qc_new)

    return obwody_po_ewie, bity_ewy, bazy_ewy
```

Atak Ewy w przypadku nieidealnego źródła BB84

- **Atak PNS (Photon Number Splitting)** - Ewa monitoruje przelatujące impulsy za pomocą zmiennej lista_multi:
 - **Kradzież** – gdy wykryje impuls wielofotonowy, zachowuje jeden foton w pamięci kwantowej, odczytując go ze 100% skutecznością po jawnym uzgodnieniu baz (w kodzie symulowane przez pobranie elementu z bity_alicji). Oryginalny obwód (qc.copy()) przelatuje nienaruszony, co **nie generuje żadnych błędów QBER**.
 - **Pominięcie** – gdy wykryje idealny impuls jednofotonowy, ignoruje go, aby uniknąć wykrycia i nie zaburzać statystyk układu.

```
# Podśluch Ewy

def podsluch_ewy(obwody_kwantowe, qrng, bity_alicji, lista_multi, strategia='losowa', tryb='probabilistyczny', prawdopodobienstwo=0.1, start_krok=0, interwal=1):
    obwody_po_ewie = []
    bity_ewy = []
    bazy_ewy = []

    for i, qc in enumerate(obwody_kwantowe):
        if strategia == 'PNS':
            if lista_multi[i] > 1:
                bity_ewy.append(bity_alicji[i])
                bazy_ewy.append(-1)
                obwody_po_ewie.append(qc.copy())
            else:
                bity_ewy.append(None)
                bazy_ewy.append(None)
                obwody_po_ewie.append(qc.copy())
            continue

        podsluchuje = False
        if tryb == 'probabilistyczny':
            if random.random() < prawdopodobienstwo: podsluchuje = True
        elif tryb == 'sekwencyjny':
            if i >= start_krok: podsluchuje = True
        elif tryb == 'interwalowy':
            if interwal > 0 and (i + 1) % interwal == 0: podsluchuje = True

        if not podsluchuje:
            obwody_po_ewie.append(qc.copy())
            bity_ewy.append(None)
            bazy_ewy.append(None)
            continue

        if strategia == 'prosta': baza = 0
        elif strategia == 'skosna': baza = 1
        else: baza = qrng.get_next_bit()

        bazy_ewy.append(baza)

        qc_ewa = qc.copy()
        if baza == 1: qc_ewa.h(0)
        wektor_stanu = Statevector(qc_ewa)
        zmierzony_wynik_str, _ = wektor_stanu.measure()
        zmierzony_bit = int(zmierzony_wynik_str)
        bity_ewy.append(zmierzony_bit)

        nowy_foton = QuantumCircuit(1)
        if zmierzony_bit == 1: nowy_foton.x(0)
        if baza == 1: nowy_foton.h(0)
        obwody_po_ewie.append(nowy_foton)

    return obwody_po_ewie, bity_ewy, bazy_ewy
```

Elementy warstwy klasycznej

- Faza **Uzgadniania baz** (funkcja `uzgodnij_klucz`):
 - Alicja i Bob publicznie porównują zestawienia użytych przez siebie **baz polaryzacji**.
 - Program iteruje przez listy baz. Jeśli były one zgodne, bity są zachowywane. Pomiar wykonany w niedopasowanych bazach (które dałyby całkowicie losowe wyniki) są trwale odrzucane.
 - Wynikiem tej funkcji jest tzw. **klucz surowy (sifted key)** – skrócony ciąg bitów, który u obu stron powinien być identyczny, pod warunkiem braku szumów środowiskowych i podsłuchu.
- Faza **Korekcji błędów** (funkcja `korekcja_bledow`):
 - **Sprawdzanie parzystości** – program dzieli surowy klucz na mniejsze segmenty (definiowane przez `rozmiar_bloku`, domyślnie 4 bity). Dla każdego bloku obliczana jest jego suma kontrolna modulo 2 ($\text{sum}(\text{blok_a}) \% 2$). Jeśli parzystość u Alicji i Boba się zgadza, blok zostaje zaakceptowany. W przypadku niezgodności, zanieczyszczony blok jest całkowicie odrzucany.
 - **Wzmocnienie prywatności** – z każdego pomyślnie zweryfikowanego bloku program celowo usuwa **ostatni bit** (instrukcja `blok[:-1]`), co uniemożliwia Ewie matematyczne odtworzenie wartości poszczególnych bitów na podstawie podsłuchanych informacji o parzystości bloku.

```
# Elementy warstwy klasycznej

def uzgodnij_klucz(bity_alicji, bazy_alicji, bity_boba, bazy_boba):
    klucz_alicji, klucz_boba = [], []
    for i in range(len(bazy_alicji)):
        if bazy_alicji[i] == bazy_boba[i]:
            klucz_alicji.append(bity_alicji[i])
            klucz_boba.append(bity_boba[i])
    return klucz_alicji, klucz_boba

def korekcja_bledow(klucz_alicji, klucz_boba, rozmiar_bloku=4):
    ostateczny_klucz_alicji, ostateczny_klucz_boba = [], []
    for i in range(0, len(klucz_alicji), rozmiar_bloku):
        blok_a = klucz_alicji[i:i+rozmiar_bloku]
        blok_b = klucz_boba[i:i+rozmiar_bloku]
        if len(blok_a) < rozmiar_bloku: continue

        if sum(blok_a) % 2 == sum(blok_b) % 2:
            ostateczny_klucz_alicji.extend(blok_a[:-1])
            ostateczny_klucz_boba.extend(blok_b[:-1])
    return ostateczny_klucz_alicji, ostateczny_klucz_boba
```

Analiza statystyczna BB84 (Atak Propabilistyczny)

RAPORT Z SYMULACJI: ATAK PROPABILISTYCZNY

Ilość fotonów w próbce: 1000 (dla każdego z punktów pomiarowych)

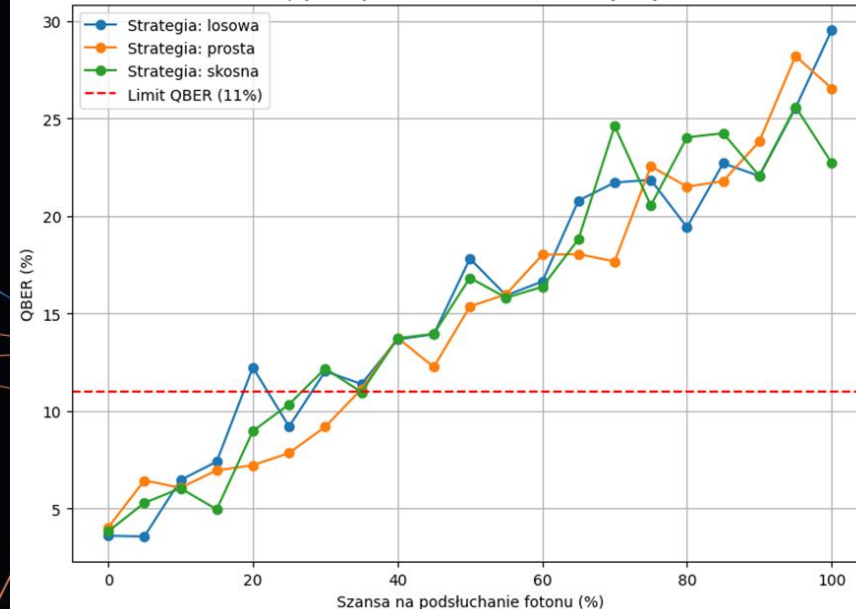
Szansa	QBER (Losowa)	QBER (Prosta)	QBER (Skośna)	Wiedza (Losowa)	Wiedza (Prosta)	Wiedza (Skośna)
0%	3.60%	4.02%	3.82%	0.00%	0.00%	0.00%
5%	3.56%	6.44%	5.28%	2.57%	3.82%	4.50%
10%	6.46%	6.06%	6.03%	7.56%	8.89%	6.47%
15%	7.40%	6.95%	4.95%	13.00%	10.71%	11.24%
20%	12.23%	7.21%	8.97%	14.40%	16.23%	16.76%
25%	9.20%	7.84%	10.34%	20.74%	20.20%	18.42%
30%	12.03%	9.19%	12.18%	23.24%	19.42%	19.16%
35%	11.38%	11.11%	10.96%	29.74%	24.17%	24.62%
40%	13.66%	13.75%	13.73%	30.43%	27.89%	25.88%
45%	13.93%	12.25%	13.95%	31.81%	30.16%	32.81%
50%	17.81%	15.36%	16.82%	38.75%	34.93%	38.32%
55%	15.92%	15.98%	15.80%	43.67%	41.42%	42.83%
60%	16.63%	18.00%	16.36%	43.12%	42.80%	41.72%
65%	20.78%	18.04%	18.80%	47.87%	47.84%	52.71%
70%	21.70%	17.66%	24.60%	49.70%	50.99%	54.40%
75%	21.84%	22.54%	20.53%	59.32%	55.13%	55.28%
80%	19.42%	21.50%	24.02%	56.70%	59.88%	58.93%
85%	22.69%	21.77%	24.23%	63.65%	64.11%	64.62%
90%	22.04%	23.81%	22.05%	64.69%	63.24%	68.11%
95%	25.53%	28.19%	25.59%	70.60%	69.55%	77.36%
100%	29.52%	26.54%	22.71%	71.81%	72.88%	75.50%

Źródło idealne

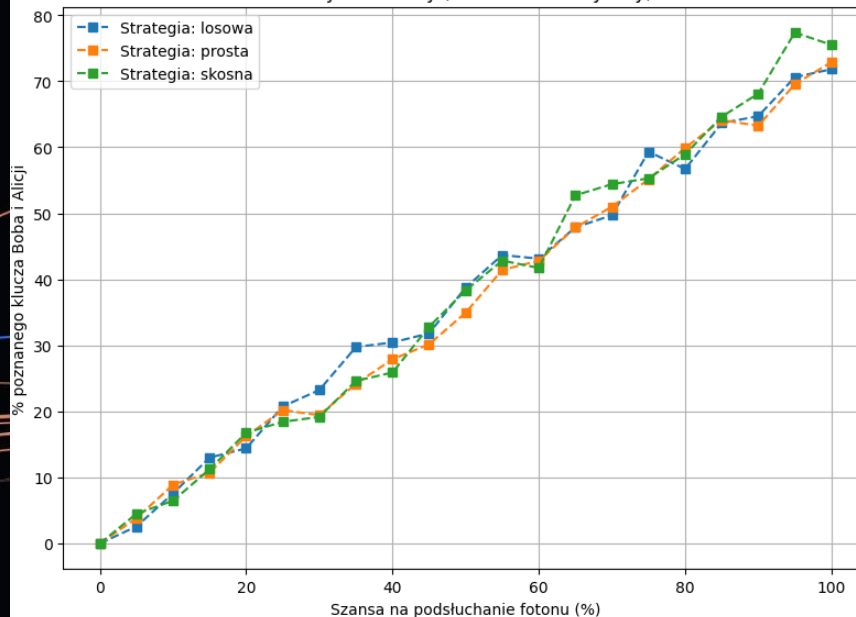
•**Teoretyczny limit QBER = 25%**, co potwierdza odczyt ~22-29% (wliczając sprzętowy szum tła) dla podsłuchu rzędu 100%.

•**Teoretyczny pułap wiedzy = 75%**, z czym zbiegają się odczyty z tabeli (~72-76%) przy podsłuchiwanie każdego fotonu.

Wpływ Ewy na QBER (Atak Probabilistyczny)



Efektywność Ewy (Atak Probabilistyczny)

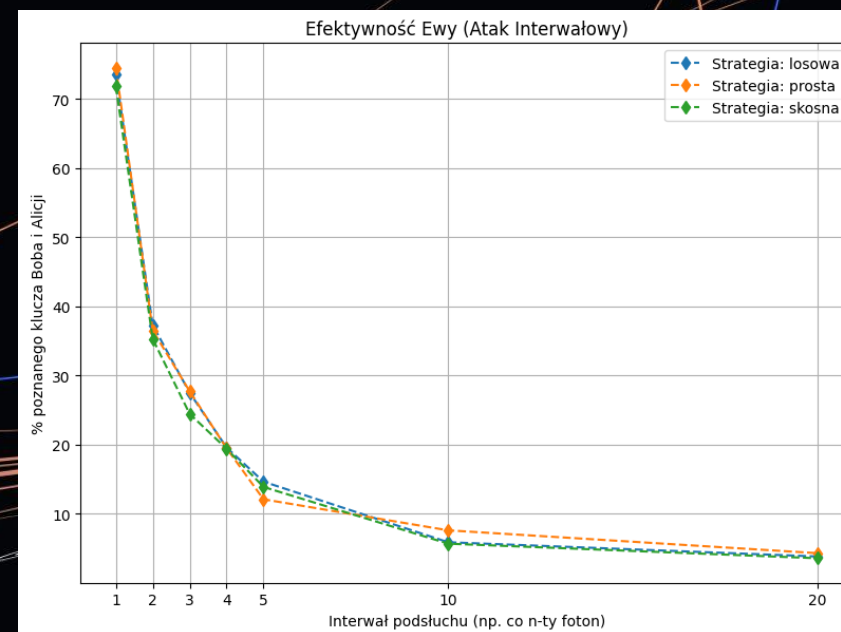
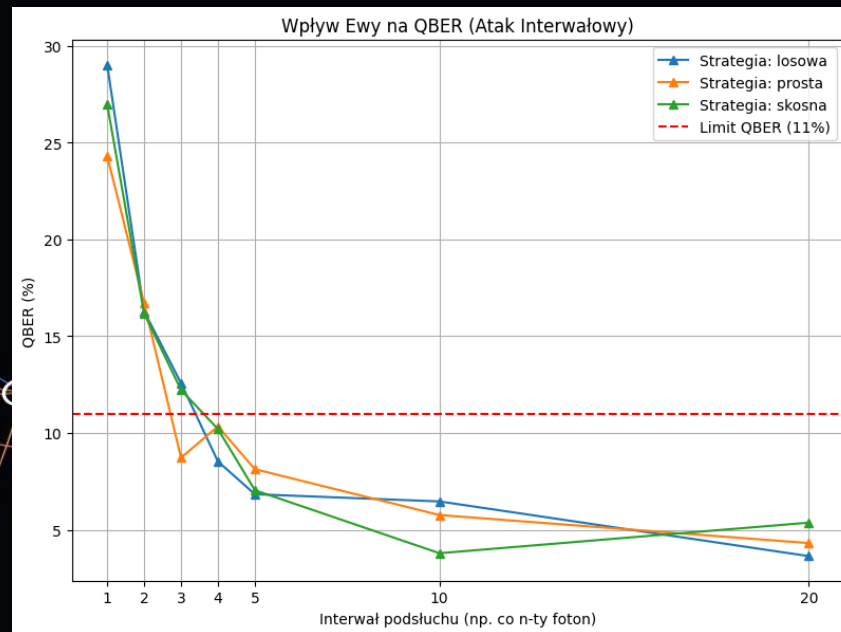


Analiza statystyczna BB84 (Atak Interwałowy)

RAPORT Z SYMULACJI: ATAK INTERWAŁOWY

Ilość fotonów w próbce: 1000 (dla każdego z punktów pomiarowych)

Interwał	QBER (Losowa)	QBER (Prosta)	QBER (Skośna)	Wiedza (Losowa)	Wiedza (Prosta)	Wiedza (Skośna)
co 1 bit	29.02%	24.30%	26.98%	73.49%	74.50%	71.81%
co 2 bit	16.29%	16.70%	16.18%	37.27%	36.49%	35.27%
co 3 bit	12.57%	8.72%	12.22%	27.50%	27.71%	24.44%
co 4 bit	8.53%	10.34%	10.20%	19.57%	19.48%	19.40%
co 5 bit	6.84%	8.13%	7.04%	14.65%	12.10%	13.88%
co 10 bit	6.46%	5.76%	3.79%	5.89%	7.61%	5.68%
co 20 bit	3.64%	4.31%	5.37%	3.83%	4.31%	3.58%



Analiza statystyczna BBM92 (Atak Probabilistyczny)

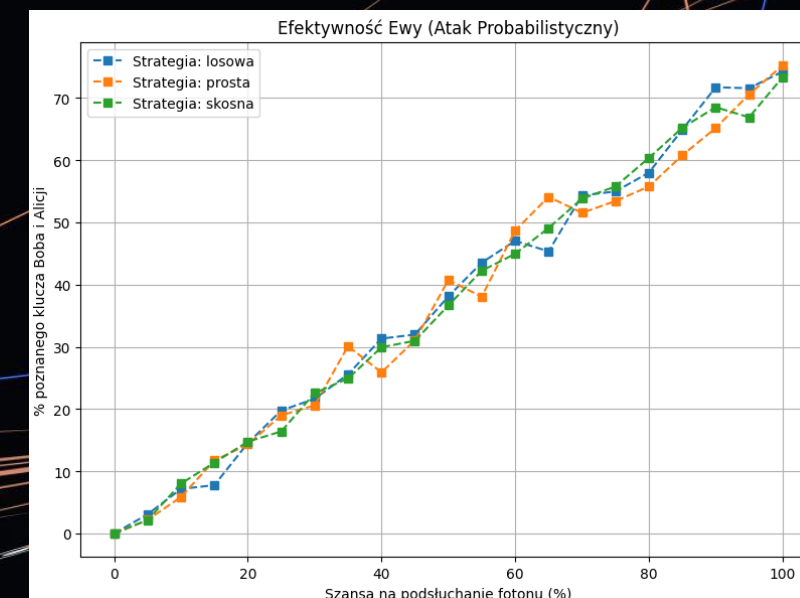
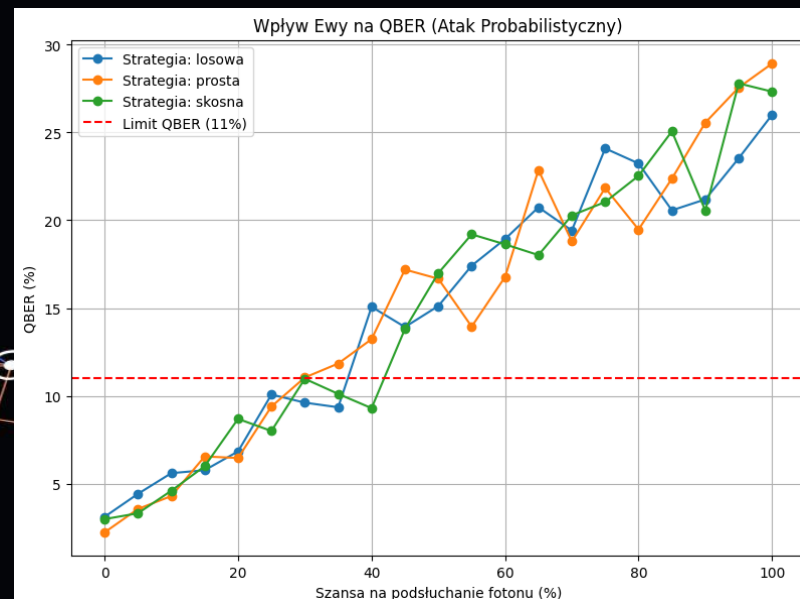
RAPORT Z SYMULACJI: ATAK PROBABILISTYCZNY

Ilość fotonów w próbce: 1000 (dla każdego z punktów pomiarowych)

Szansa	QBER (Losowa)	QBER (Prosta)	QBER (Skośna)	Wiedza (Losowa)	Wiedza (Prosta)	Wiedza (Skośna)
0%	3.12%	2.24%	2.99%	0.00%	0.00%	0.00%
5%	4.43%	3.57%	3.33%	3.08%	2.18%	2.15%
10%	5.61%	4.31%	4.61%	7.16%	5.87%	8.02%
15%	5.78%	6.56%	6.00%	7.77%	11.73%	11.39%
20%	6.85%	6.47%	8.70%	14.52%	14.31%	14.74%
25%	10.08%	9.40%	8.01%	19.76%	18.98%	16.41%
30%	9.62%	11.05%	10.98%	21.64%	20.60%	22.56%
35%	9.36%	11.84%	10.12%	25.57%	30.10%	24.90%
40%	15.07%	13.23%	9.31%	31.31%	25.88%	29.93%
45%	13.92%	17.19%	13.81%	31.96%	31.05%	30.96%
50%	15.11%	16.67%	16.98%	38.10%	40.70%	36.64%
55%	17.41%	13.94%	19.19%	43.52%	38.05%	42.22%
60%	18.92%	16.77%	18.62%	47.01%	48.67%	44.94%
65%	20.73%	22.85%	18.01%	45.33%	54.09%	49.07%
70%	19.41%	18.79%	20.27%	54.31%	51.52%	53.80%
75%	24.08%	21.83%	21.03%	54.95%	53.41%	55.75%
80%	23.22%	19.47%	22.52%	57.97%	55.74%	60.33%
85%	20.56%	22.36%	25.05%	64.87%	60.76%	65.17%
90%	21.18%	25.54%	20.55%	71.69%	65.15%	68.49%
95%	23.52%	27.54%	27.78%	71.54%	70.55%	66.87%
100%	25.99%	28.91%	27.31%	74.21%	75.20%	73.31%

Teoretyczny limit QBER = 25%, co precyzyjnie potwierdza odczyt **~25-29%** (wliczając sprzętowy szum tła) dla podsłuchu rzędu 100%.

Teoretyczny pułap wiedzy = 75%, z czym zbiegają się odczyty z tabeli (**~73-75%**).

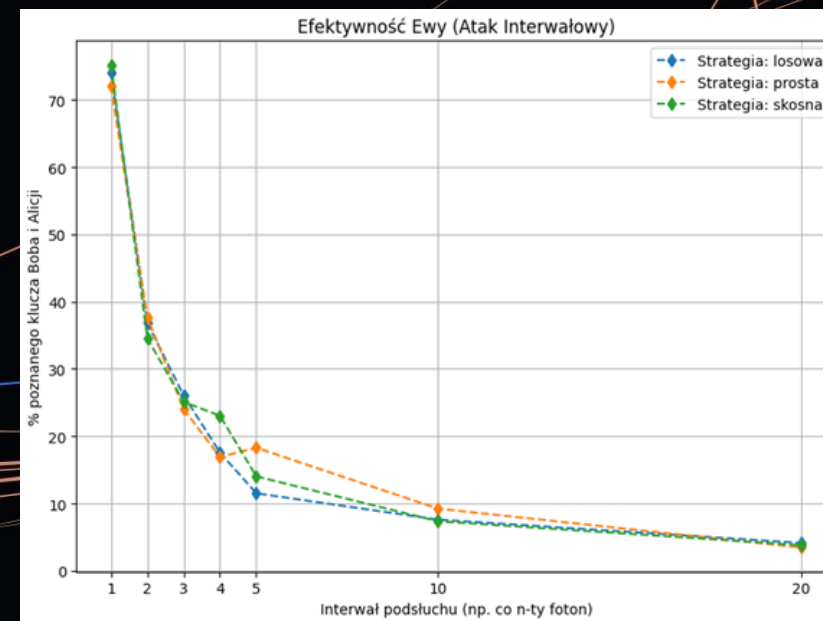
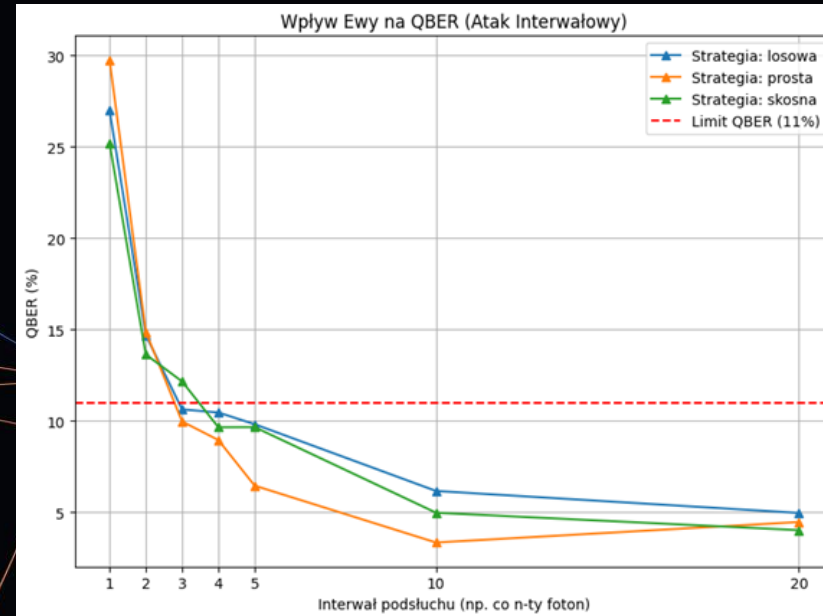


Analiza statystyczna BBM92 (Atak Interwałowy)

RAPORT Z SYMULACJI: ATAK INTERWAŁOWY

Ilość fotonów w próbce: 1000 (dla każdego z punktów pomiarowych)

Interwał	QBER (Losowa)	QBER (Prosta)	QBER (Skośna)	Wiedza (Losowa)	Wiedza (Prosta)	Wiedza (Skośna)
co 1 bit	26.98%	29.75%	25.20%	74.01%	72.02%	75.20%
co 2 bit	14.69%	14.87%	13.63%	36.82%	37.68%	34.55%
co 3 bit	10.62%	9.96%	12.15%	26.05%	24.02%	25.10%
co 4 bit	10.45%	8.95%	9.65%	17.55%	16.90%	23.03%
co 5 bit	9.81%	6.45%	9.65%	11.51%	18.28%	14.03%
co 10 bit	6.16%	3.34%	4.97%	7.60%	9.23%	7.36%
co 20 bit	4.96%	4.46%	4.02%	4.13%	3.45%	3.81%



Analiza statystyczna BB84 – nieidealne źródło

RAPORT Z SYMULACJI: WPŁYW JAKOŚCI LASERA NA ATAKI (Próbka: 1000 fotonów)

Multi-fotony	QBER (PNS)	QBER (Los)	QBER (Prost)	QBER (Skos)	Wiedza (PNS)	Wiedza (Los)	Wiedza (Prost)	Wiedza (Skos)
0%	3.45%	26.53%	28.33%	26.53%	51.06%	77.62%	74.38%	76.34%
5%	2.90%	27.14%	28.74%	27.75%	51.66%	77.55%	73.98%	76.49%
10%	2.09%	25.40%	26.97%	25.15%	56.17%	72.80%	75.39%	74.25%
15%	2.77%	26.93%	30.49%	23.49%	55.86%	73.47%	74.19%	75.68%
20%	2.37%	27.31%	26.41%	26.36%	59.88%	77.11%	72.18%	74.22%
25%	3.55%	27.90%	25.60%	26.77%	63.26%	73.93%	73.79%	75.72%
30%	3.67%	23.54%	28.37%	25.10%	62.65%	76.65%	70.82%	74.90%
35%	4.12%	26.20%	30.34%	29.24%	70.39%	76.09%	72.85%	75.26%
40%	4.26%	27.15%	27.27%	26.88%	72.13%	73.42%	74.00%	76.58%
45%	2.30%	27.03%	25.32%	27.04%	71.59%	73.60%	72.73%	77.63%
50%	4.57%	28.25%	30.66%	26.09%	74.95%	72.76%	72.95%	77.64%

Źródło nieidealne:

- **Teoretyczny przyrost QBER = 0%**, co potwierdza odczyt z tabeli rzędu ~2-5% na każdym etapie (atak nie generuje błędów, system rejestruje wyłącznie naturalny szum sprzątowy).
- **Teoretyczny pułap wiedzy rośnie liniowo** wraz z wadliwością lasera. Dla 50% impulsów wielofotonowych teoria przewiduje wiedzę Ewy rzędu **75%** (Ewa kradnie foton i w 100% zna jego bazę, a resztę zgaduje), z czym zbiegają się odczyty z tabeli (~72-78%).

